

Jetson Orin Nano Setup Guide (JetPack Install ~ Boot Logo Customization)

Target hardware: **Jetson Orin Nano Super Developer Kit** (P3767-0005 module / P3768-0000 carrier board) Installed version: **JetPack 7.2 / L4T R39.2.0** (Ubuntu 24.04 based) Host PC: Ubuntu (x86_64) End result: Replace **every NVIDIA logo with the company logo (LS Automotive)** from power-on through desktop, plus auto-login and auto-launching a GUI app.

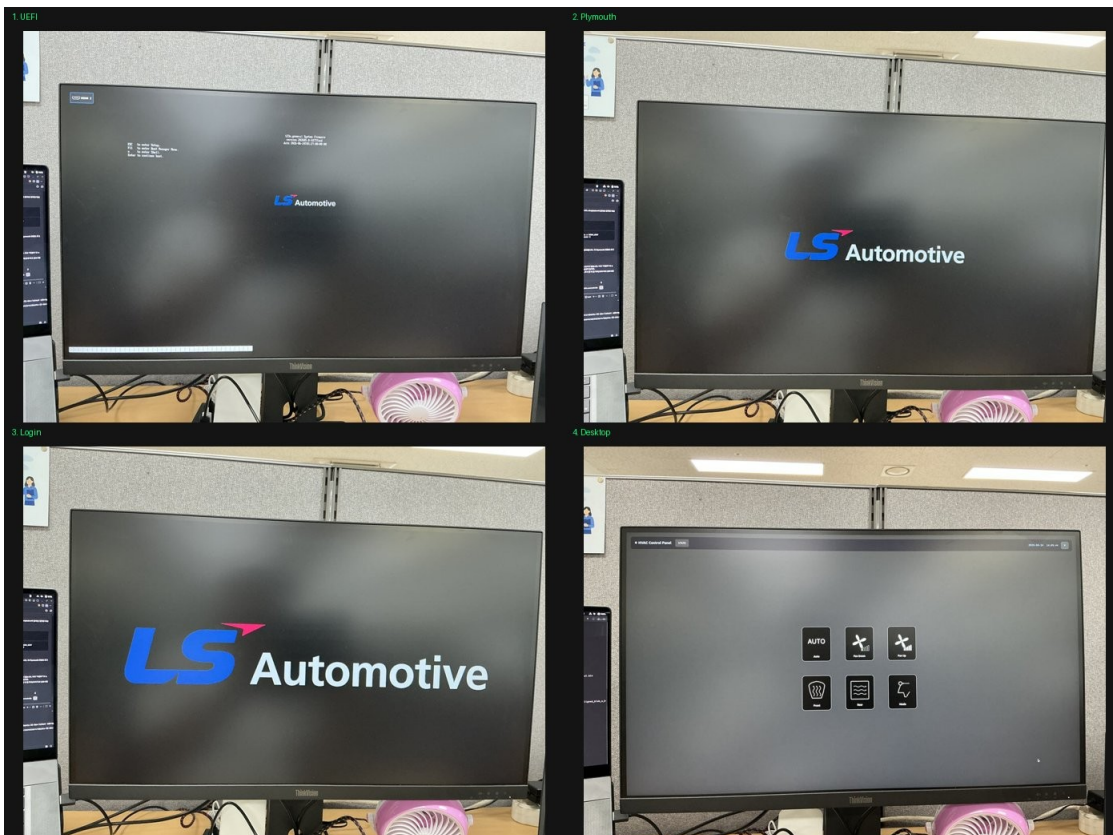


Fig. Final result — company logo at all 4 stages (UEFI / Plymouth / Login / Desktop)

0. Big Picture (4 stages where the logo appears)

On boot, the NVIDIA logo appears in **four different places**, each from a different source, so each needs a different fix.

Stage	When it shows	Source	Fix
1. UEFI firmware	Right after power-on (ESC to enter Setup screen)	BMP inside QSPI firmware	Build UEFI from source, flash QSPI
2. Plymouth	During kernel boot	Plymouth theme in rootfs	Replace theme + regenerate initrd
3. Post-login (~8s)	Right after login, before desktop	~/.xsessionrc paints it via feh	Replace background PNG
4. Desktop background	On the desktop	gsettings wallpaper	Change gsettings

1. Install JetPack 7.2 (SDK Manager)

1-1. Install SDK Manager (host PC)

1. developer.nvidia.com → search “NVIDIA SDK Manager” → log in (NVIDIA developer account) → download .deb
2. Install:

```
sudo apt install ~/Downloads/sdkmanager_*.deb
```

- o sudo: admin privileges (required to install), will prompt for password
- o apt install: Ubuntu’s package install command

1-2. Put the Jetson into Force Recovery mode

To flash, the board must be in “recovery mode”. 1. Fully power off the Jetson (disconnect power cable) 2. Connect Jetson ↔ host PC with a USB-C cable 3. **Hold the REC button** while reconnecting power (or pressing the power button) 4. Release REC after 2~3 seconds



Fig. Jetson board (underside — microSD slot / buttons / connectors)

Verify (host PC terminal):

```
lsusb | grep -i 0955
```

- 0955:7523 NVIDIA Corp. APX → recovery mode success ✓
- 0955:7020 ... L4T running on Tegra → it just booted normally (failed); retry REC timing

1-3. ⚠ SDK Manager GUI crash → work around with CLI (important pitfall)

On recent host OSes the sdkmanager GUI may crash immediately (SIGSEGV). When reinstalling/sandbox options don't help, use **CLI mode** (this is the key trick):

```
sdkmanager --query interactive --product Jetson
```

- ⚠ **Never attach a pipe like | head** — the pipe removes the TTY, breaking arrow-key menus (prompts silently hang or auto-default).
- Navigate the interactive menu with arrow keys ↑ ↓ + Enter:
 - o Action: **Install**

- o Product: Jetson
- o System configuration: **Target Hardware**
- o Target hardware: Jetson Orin Nano modules
- o SDK version: **JetPack 7.2** (show all versions: Yes)
- o Install method: **ISO Flash (Monitor)** if a monitor is attached, else **ISO Flash (Headless)**
- o Additional SDKs: select Holoscan etc. if needed

It prints a final run command (execute as-is):

```
sdkmanager --action install --login-type devzone --product Jetson \
--target-os Linux --version 7.2 --show-all-versions \
--target JETSON_ORIN_NANO_TARGETS --install-method iso_flash \
--additional-sdk 'Holoscan 4.3'
```

- Note: when the CLI's --action install reaches the actual flash step, it **relaunches the GUI itself**, and at that point the GUI works fine. (Only direct GUI launch crashed.)

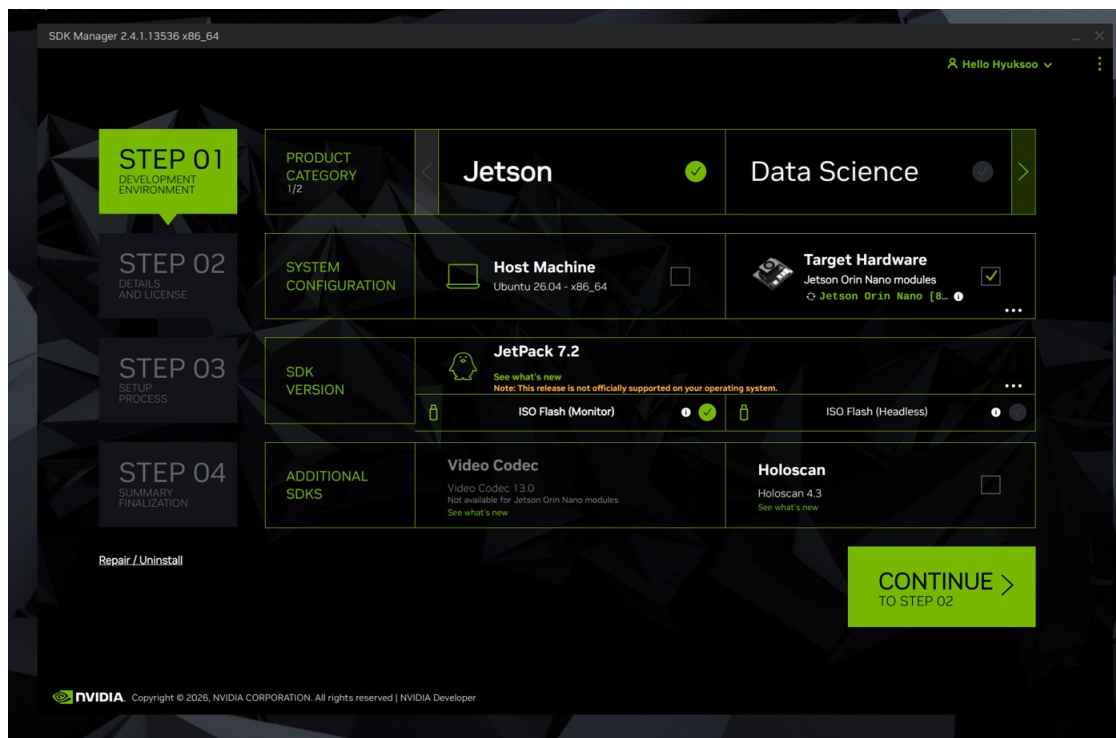


Fig. SDK Manager STEP 01 — JetPack 7.2 / Target Hardware selected

1-4. ISO flashing

- A step appears to burn the ISO onto an **empty USB stick (8GB+)** via balenaEtcher → plug in USB and proceed
- Plug that USB into the Jetson and boot → in the GRUB menu select **“Install Jetson ISO 39.2.0”** → auto-install
- **⚠ Do not remove the USB before the completion signal** — removing early hangs at EFI stub: Exiting boot services... (black screen). A power cycle recovers it, but it’s risky.

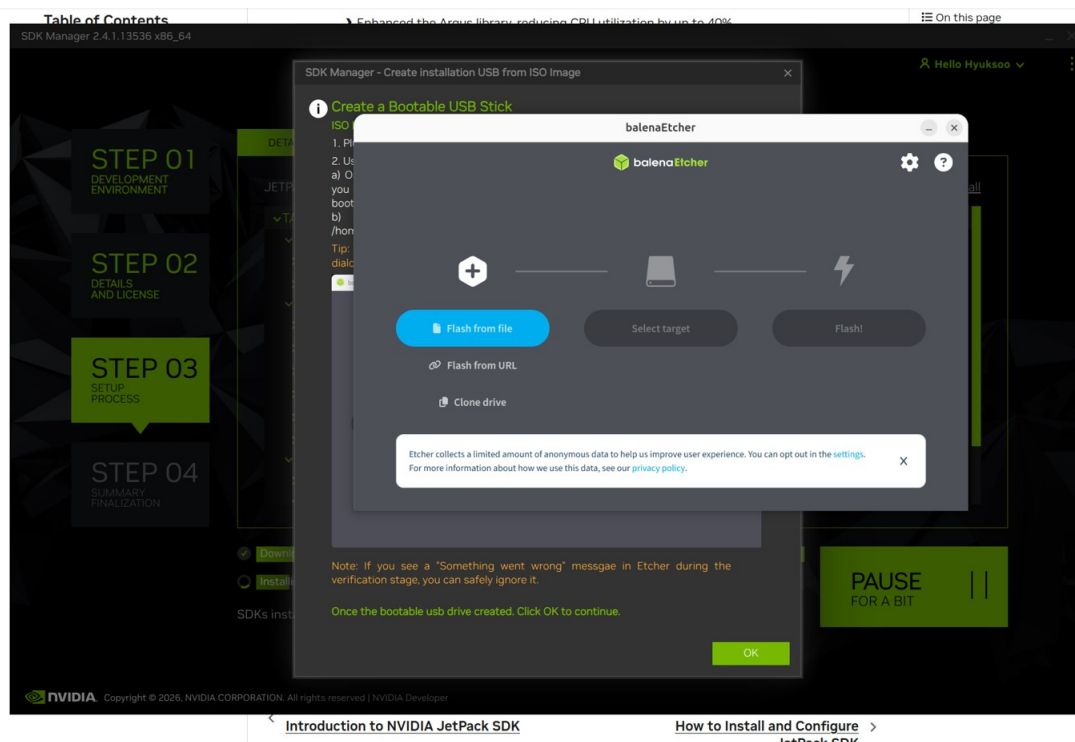


Fig. Burning the install USB with balenaEtcher 📷 [Photo: GRUB install menu “Install Jetson ISO 39.2.0” on the Jetson monitor]

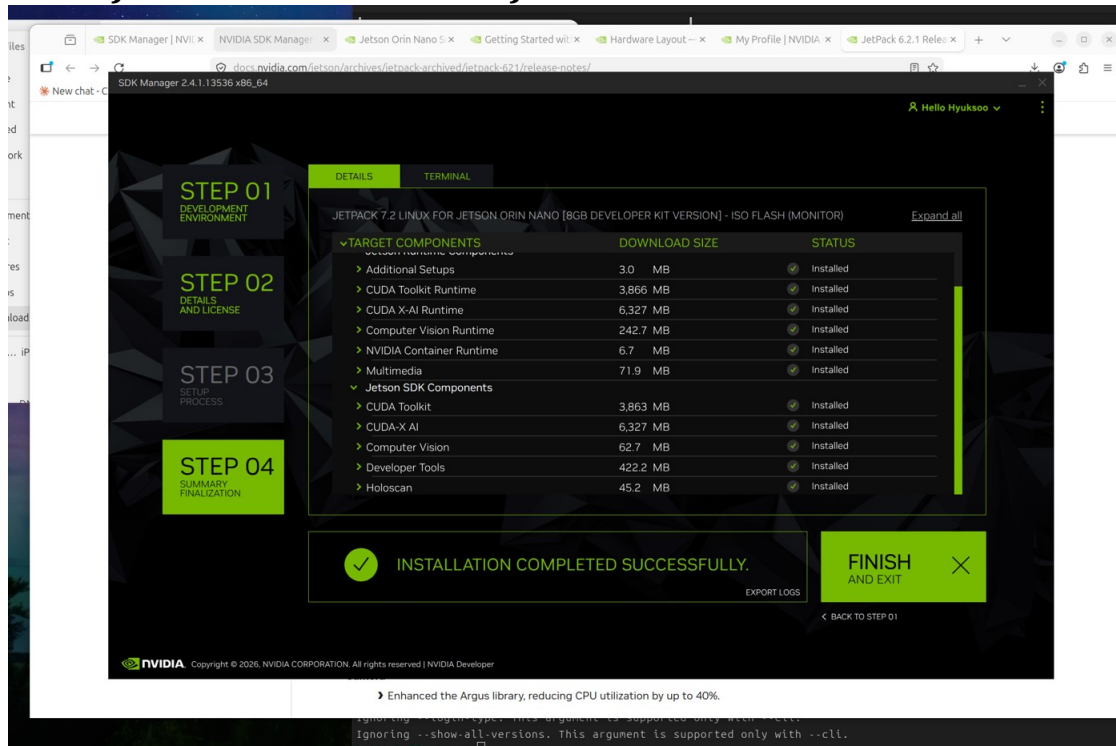


Fig. SDK Manager STEP 04 — INSTALLATION COMPLETED SUCCESSFULLY

1-5. Create account + install SDK components

- After install the Jetson reboots → Ubuntu first-boot wizard → **create an account** (e.g. hs / 0000)
 - **Skip** the “check for updates” prompt (saves time)
- In the SDK Manager dialog on the host (“SDK components install”) enter that account + USB connection (IP 192.168.55.1) → CUDA/Holoscan etc. install automatically
- Done when it shows “INSTALLATION COMPLETED SUCCESSFULLY”

2. SSH into the Jetson from the host PC (work convenience)

You can work over the USB network without pulling the SD card each time.

check connectivity

`ping -c 2 192.168.55.1`

register SSH key (password-less login, one time)

```
ssh-copy-id hs@192.168.55.1 # enter password 0000
```

```
# connect
```

```
ssh hs@192.168.55.1
```

- Afterward you can run remote commands from the host: `ssh hs@192.168.55.1 "command"`
 - For admin commands, pipe the password: `echo 0000 | sudo -S command`
-

3. Logo Replacement ① — UEFI firmware (most complex, source build required)

The logo shown right after power-on. It's baked into the QSPI firmware as a BMP, so you must **build UEFI from source**.

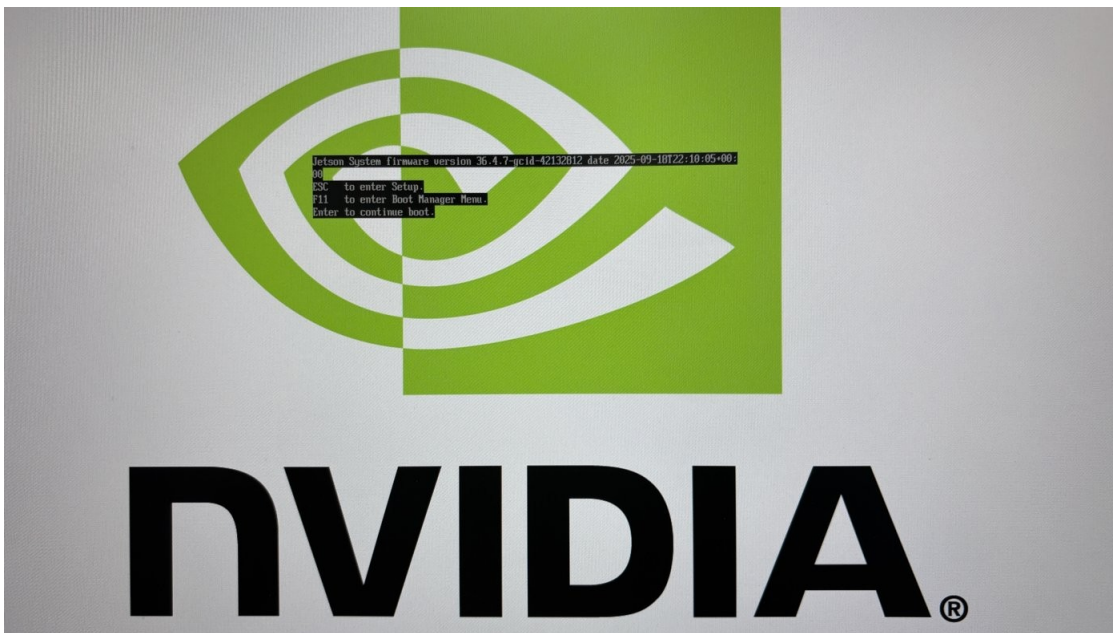


Fig. (Before) UEFI stage — NVIDIA logo + firmware version

3-1. Docker build environment

```
sudo apt install docker.io
```

```
sudo usermod -a -G docker $USER # add current user to the docker group
```

```
# → applies after logout/reboot (otherwise run with sudo docker)
```

3-2. Clone the UEFI source workspace

```
sudo mkdir -p /build && sudo chown $USER:$USER /build
```

git identity is required (edkrepo clone fails without it)

```
git config --global user.name "yourname"
git config --global user.email "you@example.com"
```

IMG=ghcr.io/tianocore/containers/ubuntu-22-dev:latest

init edkrepo config

```
sudo docker run --rm -v "$HOME":"$HOME" -e EDK2_DOCKER_USER_HOME="$HOME"
$IMG init_edkrepo_conf
sudo docker run --rm -v "$HOME":"$HOME" -e EDK2_DOCKER_USER_HOME="$HOME"
$IMG \
    edkrepo manifest-repos add nvidia https://github.com/NVIDIA/edk2-edkrepo-
manifest.git main nvidia
```

clone workspace (~400k objects; retry on an empty dir if the internet drops)

```
cd /build
sudo docker run --rm -w /build -v /build:/build -v "$HOME":"$HOME" \
    -e EDK2_DOCKER_USER_HOME="$HOME" $IMG \
    edkrepo clone nvidia-uefi NVIDIA-Platforms main
```

3-3. ⚠️ Replace the *actual* logo file (key pitfall)

It is **LogoSingleBlack**, not `nvidiagray*.bmp` (`LogoMultipleGray`), that is actually used. - Confirm in `/build/nvidia-uefi/nvidia-config/t23x_general/.config:`
`CONFIG_LOGO_IMPLEMENTER="Silicon/NVIDIA/Drivers/Logo/LogoSingleBlack.inf"` - File to replace: `Silicon/NVIDIA/Drivers/Logo/nvidiablack-1036x864.bmp` (1036×864, **black background**)

Create the company logo at the same spec (1036×864, black bg, 24-bit BMP) and overwrite:

Python (PIL) example — company logo PNG → 1036x864 black-bg BMP

```
from PIL import Image
src = Image.open('company_logo.png').convert('RGBA')
w, h = 1036, 864
canvas = Image.new('RGB', (w, h), (0,0,0))
tw = int(w*0.5); s = tw/src.width; th = int(src.height*s)
logo = src.resize((tw, th), Image.LANCZOS)
canvas.paste(logo, ((w-tw)//2, (h-th)//2), logo)
canvas.save('/build/nvidia-uefi/edk2-nvidia/Silicon/NVIDIA/Drivers/Logo/
nvidiablack-1036x864.bmp', 'BMP')
```


3-4. Build UEFI

```
cd /build/nvidia-uefi
sudo docker run --rm -w /build/nvidia-uefi -v /build:/build -v "$HOME":"$HOME" \
-e EDK2_DOCKER_USER_HOME="$HOME" $IMG \
edk2-nvidia/Platform/NVIDIA/Tegra/build.sh \
--init-defconfig edk2-nvidia/Platform/NVIDIA/Tegra/DefConfigs/t23x_general.defconfig
```

- Output: /build/nvidia-uefi/images/uefi_t23x_general_DEBUG.bin (~3MB; must be < 3.5MB UEFI partition)

3-5. Flash to QSPI (BSP required)

1. developer.nvidia.com → Jetson Linux 39.2 → **Driver Package (BSP)**
download (Jetson_Linux_R39.2.0_aarch64.tbz2, NOT Sources!)

2. Extract and swap in your built binary:

```
tar xjf ~/Downloads/Jetson_Linux_R39.2.0_aarch64.tbz2 -C /tmp/
cp /build/nvidia-uefi/images/uefi_t23x_general_DEBUG.bin \
/tmp/Linux_for_Tegra/bootloader/uefi_bins/uefi_t23x_general.bin
sudo apt install -y libxml2-utils #flash.sh needs xmllint
```

3. Put the Jetson into **recovery mode** (see 1-2)
4. ⚠ Flash **both A and B slots** (flashing only A leaves it booting from B with no effect):

```
cd /tmp/Linux_for_Tegra
# slot A
sudo ./flash.sh -k A_cpu-bootloader -c bootloader/generic/cfg/flash_t234_qspi.xml \
jetson-orin-nano-devkit-super mmcblk0p1
# → re-enter recovery mode, then
# slot B
sudo ./flash.sh -k B_cpu-bootloader -c bootloader/generic/cfg/flash_t234_qspi.xml \
jetson-orin-nano-devkit-super mmcblk0p1
```

- o This writes **only a partition** → the SD card's OS is not wiped

📷 [Photo: flash.sh output “The [A_cpu-bootloader] has been updated successfully”]

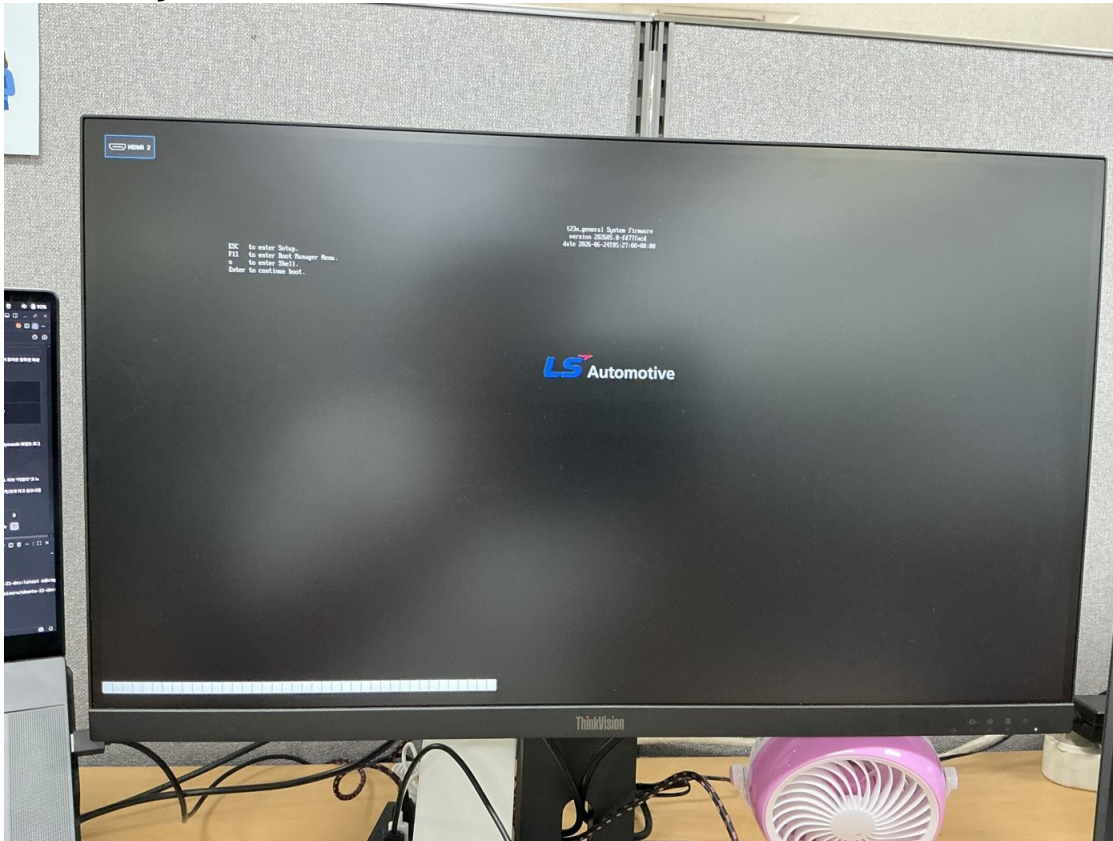


Fig. (After) UEFI stage — company logo

4. Logo Replacement ② — Plymouth boot splash

The logo during kernel boot. (All of the below runs on the Jetson directly or via SSH.)

4-1. Create the custom theme

```
sudo mkdir -p /usr/share/plymouth/themes/company-logo
```

Prepare logo.png — **black background, ~480px wide** (the real Plymouth surface is ~1200px, so $480\text{px} \approx 40\%$ of screen):

```
from PIL import Image
src = Image.open('company_logo.png').convert('RGBA')
tw = 480; s = tw/src.width; th = int(src.height*s)
logo = src.resize((tw, th), Image.LANCZOS)
canvas = Image.new('RGB', (tw, th), (0,0,0))
```

```
canvas.paste(logo, (0,0), logo)
canvas.save('logo.png')
```

💡 Plymouth's built-in scaler is low quality → **pre-downscale to the exact size with PIL (LANCZOS) and place natively (no scaling) in the script** for the sharpest result. Note: since the Plymouth framebuffer is ~1200px and the monitor upscales ~2x, it can't physically be as sharp as the login logo (rendered at native 2560).

company-logo.plymouth:

[Plymouth Theme]

Name=Company Logo

ModuleName=script

[script]

ImageDir=/usr/share/plymouth/themes/company-logo

ScriptFile=/usr/share/plymouth/themes/company-logo/company-logo.script

company-logo.script (center, no scaling):

```
Window.SetBackgroundTopColor(0, 0, 0);
Window.SetBackgroundBottomColor(0, 0, 0);
logo.image = Image("logo.png");
logo.sprite = Sprite(logo.image);
logo.sprite.SetX(Window.GetWidth() / 2 - logo.image.GetWidth() / 2);
logo.sprite.SetY(Window.GetHeight() / 2 - logo.image.GetHeight() / 2);
fun refresh_callback() { }
Plymouth.SetRefreshFunction(refresh_callback);
```

4-2. Activate the theme

```
sudo ln -sf /usr/share/plymouth/themes/company-logo/company-logo.plymouth \
/etc/alternatives/default.plymouth
echo -e "[Daemon]\nTheme=company-logo" | sudo tee /etc/plymouth/plymouthd.conf
```

4-3. ⚠️ Boot option + initrd pitfalls (the crux)

With default settings Plymouth **won't show** on this image. Three things must be fixed:

(a) Kernel boot options — add splash quiet console=tty0 to the APPEND line in /boot/extlinux/extlinux.conf:

```
sudo cp /boot/extlinux/extlinux.conf /boot/extlinux/extlinux.conf.bak
sudo sed -i 's/console=ttyTCU0,115200/console=ttyTCU0,115200 console=tty0/'
```

```
/boot/extlinux/extlinux.conf
```

```
sudo sed -i 's/video=efib:off/video=efib:off splash quiet/' /boot/extlinux/extlinux.conf
```

Without console=tty0, Plymouth thinks there's "no display" and falls back to an empty text theme (required!)

(b) --graphical-boot option — add it to the initramfs script and the systemd units:

```
# initramfs script
```

```
sudo sed -i 's/--pid-file=/run/plymouth/pid#--pid-file=/run/plymouth/pid --graphical-boot#'
```

```
\
```

```
/usr/share/initramfs-tools/scripts/init-premount/plymouth
```

```
# systemd drop-ins (boot / reboot / shutdown screens)
```

```
for svc in plymouth-start plymouth-reboot plymouth-poweroff; do
```

```
  sudo mkdir -p /etc/systemd/system/$svc.service.d
```

```
  mode=boot; [ "$svc" = plymouth-reboot ] && mode=reboot; [ "$svc" = plymouth-  
poweroff ] && mode=shutdown
```

```
  printf '[Service]\nExecStart=\nExecStart=/usr/sbin/plymouthd --mode=%s --attach-to-  
session --graphical-boot\n' "$mode" \
```

```
  | sudo tee /etc/systemd/system/$svc.service.d/override.conf
```

```
done
```

```
sudo systemctl daemon-reload
```

Without --graphical-boot, plymouthd refuses the graphical splash because the console isn't a VT

(c) Regenerate initrd — ⚠ update-initramfs -u fails with "Nothing to do". You must use mkinitramfs -o directly:

```
sudo mkinitramfs -o /boot/initrd $(uname -r)
```

Reboot and the Plymouth company logo appears.



Fig. Plymouth boot splash — company logo

5. Logo Replacement ③ — the post-login ~8s logo (.xsessionrc)

A full-screen NVIDIA logo for ~8s right after login, before the desktop appears.

The cause is ~/.xsessionrc:

```
# this line paints the NVIDIA logo as the background via feh  
feh --no-fehbg --bg-scale /usr/share/backgrounds/NVIDIA_Login_Logo.png
```

The white-background logo = this file. (The decisive clue distinguishing it from the black-background UEFI logo.)

Fix: replace that PNG with the company logo (1920×1080; black bg recommended for consistency):

```
from PIL import Image  
src = Image.open('company_logo.png').convert('RGBA')  
w, h = 1920, 1080
```



```
canvas = Image.new('RGB', (w, h), (0,0,0)) # black background
tw = int(w*0.4); s = tw/src.width; th = int(src.height*s)
logo = src.resize((tw, th), Image.LANCZOS)
canvas.paste(logo, ((w-tw)//2, (h-th)//2), logo)
canvas.save('NVIDIA_Login_Logo.png')
```

```
sudo cp /usr/share/backgrounds/NVIDIA_Login_Logo.png
/usr/share/backgrounds/NVIDIA_Login_Logo.png.bak
sudo cp NVIDIA_Login_Logo.png /usr/share/backgrounds/NVIDIA_Login_Logo.png
```

⚠ A sub-0.5s flicker may appear at the `simplifiedrm→nvidia-drm` handoff; this is closed NVIDIA driver behavior and is not fixable. Ignore it.



Fig. Post-login transition — company logo



Fig. Final desktop — GUI app (HVAC panel) auto-launched

6. Finishing touches (desktop wallpaper / auto-login / GUI app autostart)

6-1. Desktop + lock screen wallpaper

```
sudo cp company_bg.jpg /usr/share/backgrounds/company-bg.jpg
gsettings set org.gnome.desktop.background picture-uri
'file:///usr/share/backgrounds/company-bg.jpg'
gsettings set org.gnome.desktop.background picture-uri-dark
'file:///usr/share/backgrounds/company-bg.jpg'
gsettings set org.gnome.desktop.background picture-options 'zoom'
# create a marker so NVIDIA's default-background script won't overwrite it
mkdir -p ~/.local/share/applications && touch
~/.local/share/applications/nvbackground_ubuntu
```

6-2. Auto-login — /etc/gdm3/custom.conf

```
sudo sed -i 's/# AutomaticLoginEnable = true/AutomaticLoginEnable = true/'  
/etc/gdm3/custom.conf  
sudo sed -i 's/# AutomaticLogin = user1/AutomaticLogin = hs/' /etc/gdm3/custom.conf
```

6-3. GUI app autostart — ~/.config/autostart/

```
mkdir -p ~/.config/autostart  
cat > ~/.config/autostart/touch_panel.desktop <<'EOF'  
[Desktop Entry]  
Type=Application  
Name=Jetson Touch Panel  
Exec=/home/hs/jetson_touch_c/touch_panel  
Path=/home/hs/jetson_touch_c  
X-GNOME-Autostart-enabled=true  
EOF
```

💡 An app using GPIO/SPI runs without sudo if the account is in the gpio group (check with groups). If a leftover root-launched process is holding GPIO lines, you get [GPIO] line N output request failed → clean it with sudo pkill -9 appname.

7. Cloning the SD card (for mass-producing identical boards)

You can copy this configured SD card to other identical boards. - Image the whole card with dd or balenaEtcher and write it to a new SD card (same or larger capacity) □ behaves identically - ⚠️ If multiple boards are **on the same network simultaneously**, identical hostname/SSH keys collide → change with sudo hostnamectl set-hostname newname on each board - The UEFI (QSPI) firmware lives on the board, not the SD card, so **each new board must be UEFI-flashed (section 3) separately** for the UEFI logo to change. (SD card cloning only carries the Plymouth/login/desktop logos.)

8. Pitfalls Summary (the time sinks)

Pitfall	Fix
SDK Manager GUI crashes immediately	Work around with CLI mode (--query interactive), no pipes

Pitfall	Fix
UEFI logo won't change	It's not LogoMultipleGray — the real one is LogoSingleBlack/nvidiabla ck-1036x864.bmp
Flashed but logo unchanged	Must flash both A and B slots
Plymouth won't show	Needs both console=tty0 and --graphical-boot
initrd not updating	Use mkinitramfs -o , not update-initramfs -u
NVIDIA logo after login	Not GDM/extensions — it's feh in ~/.xsessionrc
Plymouth logo blurry/wrong size	Pre-downscale to ~480px with PIL + native placement (avoid Plymouth's scaler)

Authored: 2026-06-24 / JetPack 7.2 (L4T R39.2.0) / Jetson Orin Nano Super Devkit